

LBG Express Colis — guide d'exploitation

Ce document permet de faire tourner, modifier et dépanner le site

<https://www.lbgexpresscolis.fr> sans dépendre d'aucune plateforme externe.

Tout ce qui fait vivre le site (code, base, sauvegardes, envoi d'e-mails, paiements, chat) tourne sur le serveur de l'entreprise ou sur des comptes qui lui appartiennent.

1. Où se trouve quoi

Élément	Emplacement
Serveur	VPS Hostinger, <code>187.7.18.167</code> (Ubuntu). Accès : hPanel Hostinger → VPS → Console Web, ou SSH <code>root@187.7.18.167</code>
Code du site en production	<code>/var/www/lbg-express</code>
Base de données (SQLite)	<code>/var/lib/lbg-express/lbg.db</code>
Documents envoyés par les livreurs	<code>/var/lib/lbg-express/uploads/</code>
Configuration et clés API	<code>/var/www/lbg-express/.env</code> (lisible par <code>root</code> uniquement)
Sauvegardes	<code>/var/backups/lbg-express/</code>
Journal du site	<code>/var/log/lbg-express.log</code>
Configuration du serveur web	<code>/etc/nginx/sites-enabled/lbg-express</code>
Certificat HTTPS	Let's Encrypt, renouvelé tout seul (<code>certbot.timer</code>)

Le site est un service systemd nommé `lbg-express` : il écoute en interne sur le port 4200 et nginx l'expose en HTTPS sur le domaine.

```
systemctl status lbgi-express      # état
systemctl restart lbgi-express     # redémarrer
tail -f /var/log/lbgi-express.log # voir ce qui se passe en direct
```

2. Les services extérieurs utilisés (et qui les possède)

Service	À quoi il sert	Compte
OpenAI	Chat d'assistance du site	Compte OpenAI de l'entreprise (<code>OPENAI_API_KEY</code>)
Google Analytics 4	Statistiques de visite	Compte Google de l'entreprise (<code>VITE_GA4_MEASUREMENT_ID</code>)
Google Maps	Autocomplétion des adresses	<code>VITE_GOOGLE_MAPS_API_KEY</code>
Resend	E-mails (devis, suivi, alertes serveur)	<code>RESEND_API_KEY</code>
myPOS	Paieinent en ligne	<code>MYPOS_*</code>
Hostinger	Serveur et domaine	hPanel

Aucun de ces services n'est intermédié : les clés sont celles de l'entreprise, et une facture ou un accès perdu ne dépend de personne d'autre.

3. Variables de configuration

Elles vivent **uniquement** dans `/var/www/lbgi-express/.env` sur le serveur.

Le fichier n'est jamais écrasé par un déploiement. `.env.example` du dépôt en liste la totalité avec des explications.

Après toute modification du `.env` :

```
systemctl restart lbgi-express
```

Attention : les variables préfixées `VITE_` sont lues **au moment du build**, pas au démarrage. Les changer impose de relancer un déploiement (voir §4), pas seulement un redémarrage.

Changer le modèle du chat IA

```
nano /var/www/lbg-express/.env      # AI_MODEL="gpt-5.4-mini"
systemctl restart lbg-express
```

Tout modèle du compte OpenAI fonctionne (`gpt-4o` , `gpt-5` , `gpt-5.4` ...).

Le modèle par défaut si la variable est absente est `gpt-5.4-mini` .

Si `OPENAI_API_KEY` manque, le site fonctionne normalement mais le chat répond poliment qu'il est indisponible — rien d'autre ne casse.

Pour changer de fournisseur (Anthropic, Mistral...), un seul fichier est à remplacer : `packages/web/src/api/agent/gateway.ts` .

Changer l'identifiant Google Analytics

Modifier `VITE_GA4_MEASUREMENT_ID` dans le `.env` , puis **redéployer**.

Le script de mesure n'est injecté que si la variable existe, et il respecte le bandeau de cookies du site : aucune mesure avant acceptation (Consent Mode v2).

4. Mettre le site à jour (déploiement)

Depuis le poste de travail où se trouve le code :

```
bash ops/deploy.sh
```

Le script, dans l'ordre : sauvegarde la base, envoie le code (sans les secrets, sans la base, sans les documents livreurs), installe les dépendances, construit le site, met le schéma de base à jour, redémarre le service et vérifie que la page d'accueil répond `200` . Il s'arrête et affiche l'erreur si une étape échoue.

Vérifier les valeurs en tête du script avant le premier usage : `APP` (chemin du code local), `KEY` (clé SSH) et `HOST` .

5. Sauvegardes et restauration

Une sauvegarde part **chaque nuit à 03h30** et avant chaque déploiement. Elle contient la base de données, les documents des livreurs et la configuration. Les sauvegardes de plus de 14 jours sont supprimées.

```
systemctl list-timers lbg-backup.timer    # prochaine exécution
/usr/local/bin/lbg-backup.sh             # en lancer une tout de suite
ls -la /var/backups/lbg-express/         # ce qui existe
tail /var/log/lbg-backup.log              # historique
```

Restaurer la base

```
systemctl stop lbg-express
cp /var/lib/lbg-express/lbg.db /var/lib/lbg-express/lbg.db.avant-restauration
gunzip -c /var/backups/lbg-express/lbg-2026-09-12-0330.db.gz > /var/lib/lbg-express/lbg.db
chown root:root /var/lib/lbg-express/lbg.db
systemctl start lbg-express
```

Restaurer les documents des livreurs

```
tar xzf /var/backups/lbg-express/uploads-2026-09-12-0330.tar.gz -C /var/lib/lbg-express
```

Copie hors du serveur (recommandé)

Les sauvegardes sont sur le même disque que le site : elles protègent d'une fausse manœuvre, pas de la perte du serveur. Pour en garder une copie ailleurs, depuis un poste local :

```
scp -i ~/.ssh/lbg_vps root@187.7.18.167:/var/backups/lbg-express/* ./sauvegardes-lbg/
```

À faire une fois par mois, ou automatiser depuis un poste allumé en permanence. Les fichiers `config-*.env` contiennent les clés API : ne pas les laisser traîner sur une clé USB ou un dossier partagé.

6. Surveillance automatique

`lbg-watch.timer` interroge le site toutes les 5 minutes. En cas de panne il réessaie, redémarre le service et envoie un e-mail à l'adresse `MAIL_OPS` . Un second e-mail confirme le rétablissement.

```
tail /var/log/lbg-watch.log
```

7. Travailler sur le code

```
bun install      # une fois
bun run dev      # site en local sur http://localhost:4200
bun run lint
bun run typecheck
bun run build:web
```

Structure : tout le site est dans `packages/web` — l'API (Hono) dans `src/api` , l'interface (React) dans `src/web` , le schéma de base (Drizzle) dans `src/api/database` .

Les dossiers `packages/mobile` (Expo) et `packages/desktop` (Electron) sont des restes du modèle de projet : ils ne sont **pas** utilisés, **pas** déployés et **pas** installés en production. Le déploiement les exclut explicitement. Ils peuvent rester là sans conséquence.

Base de données en local

`DATABASE_URL` peut pointer vers un fichier SQLite local (`file:./local.db`). Le site crée le fichier et les tables tout seul au premier démarrage si la base est vide ; si le chemin n'est pas accessible en écriture, il bascule automatiquement sur un emplacement utilisable au lieu de refuser de démarrer.

Après modification du schéma :

```
cd packages/web && bun run db:push
```

8. Dépannage

Symptôme	À faire
Le site ne répond pas	<code>systemctl status lb-g-express</code> puis <code>tail -50 /var/log/lb-g-express.log</code>
Erreur après un déploiement	<code>journalctl -u lb-g-express -n 50</code> . Le code précédent est récupérable depuis Git
Le chat ne répond plus	Vérifier le crédit du compte OpenAI et la validité de <code>OPENAI_API_KEY</code>
Plus aucune statistique	Vérifier <code>VITE_GA4_MEASUREMENT_ID</code> puis redéployer (variable de build)
Les e-mails ne partent plus	Vérifier <code>RESEND_API_KEY</code> et le domaine vérifié chez Resend
HTTPS expiré	<code>certbot renew --dry-run</code> puis <code>systemctl reload nginx</code>
Disque plein	<code>du -sh /var/log/* /var/backups/*</code> ; les journaux sont tournés par <code>logrotate</code>

9. Ce dont le site n'a plus besoin

Le site a été rendu indépendant de la plateforme sur laquelle il a été développé :

- le chat appelle OpenAI directement, sans passerelle intermédiaire ;
- les statistiques vont dans Google Analytics 4, sur le compte de l'entreprise ;
- le site publié ne contient plus aucun code de la plateforme (l'outil d'annotation visuelle est remplacé par un composant vide au moment du build de production) ;
- la configuration du serveur ne contient plus aucune clé de la plateforme.

Le code peut donc être repris par n'importe quel développeur : `bun install` , `bun run dev` , et tout fonctionne.